Ministère de l'Enseignement
Supérieur et de la Recherche
Scientifique

Année Universitaire
2025/2026

EPI DIGITAL SCHOOL
Ecole Internationale Supérieure
Privée Polytechnique de Sousse

RAPPORT DE STAGE PROFESSIONNEL

Spécialité: .....

Intitulé

.....

Lieu du stage **Réalisé par** **Encadré par**

Remerciements

Résumé

Ce rapport présente la conception et la réalisation, durant un stage professionnel d'un mois, d'une application web interne dédiée à la gestion des demandes techniques. Le projet couvre l'enregistrement, la qualification, l'affectation, le traitement, la résolution et la clôture des demandes, avec une historisation des changements importants. L'architecture associe un frontend Angular, une API REST Spring Boot et une base PostgreSQL, selon une organisation modulaire séparant présentation, application, domaine et persistance. La sécurité repose sur l'authentification JWT, le contrôle d'accès fondé sur les rôles et des contrôles métier contextuels. Une assistance IA intégrée via Groq propose une catégorie et une priorité à partir du titre et de la description ; le Responsable technique conserve la décision finale. La validation s'appuie sur des tests automatisés backend et frontend, une intégration locale du parcours IA et une vérification fonctionnelle humaine. Le rapport distingue les résultats mesurés sur le périmètre V1 des limites, notamment le caractère exploratoire du benchmark IA et l'absence de portail client.

Mots-clés : génie logiciel ; API REST ; Angular ; Spring Boot ; intelligence artificielle.

Abstract

This report presents the design and implementation of an internal web application for managing technical requests. Developed during a one-month professional internship, the solution addresses a controlled internal scope. The project covers request registration, qualification, assignment, processing, resolution and closure, with the history of significant changes. The architecture combines an Angular frontend, a Spring Boot REST API and a PostgreSQL database, using modular responsibilities for presentation, application, domain and persistence. Security relies on JWT authentication, role-based access control and contextual business checks. An AI assistance feature integrated through Groq proposes a category and a priority from the request title and description ; the Technical Manager remains responsible for the final decision. Validation combines automated backend and frontend tests, a local integration of the AI workflow and human functional verification. The report distinguishes measured results within the V1 scope from remaining limitations, including the exploratory nature of the AI benchmark and the absence of a customer portal.

Keywords : software engineering ; REST API ; Angular ; Spring Boot ; artificial intelligence.

Sommaire

Liste des figures **iv**

Liste des tableaux **v**

Liste des acronymes **vi**

Introduction générale **1**

Chapitre 1 — Contexte et cadrage du projet **2**
1.1 Organisme d'accueil et cadre du stage 2
1.2 Contexte, existant et problématique 3
1.3 Solution proposée, objectifs et périmètre 3
1.4 Méthodologie et planification du travail 4

Chapitre 2 — Analyse et conception **5**
Introduction 5
2.1 Spécification des besoins 5
2.1.1 Acteurs et responsabilités 5
2.1.2 Besoins fonctionnels 5
2.1.3 Besoins non fonctionnels 6
2.2 Conception fonctionnelle 6
2.3 Conception technique 9
2.3.1 Architecture générale 9
2.3.2 Principes de conception et patterns 9
2.3.3 Sécurité de la solution 12
2.3.4 Assistance intelligente 12
2.4 Conception de la base de données 12
Conclusion 14

Chapitre 3 — Réalisation et validation **15**
3.1 Vue d'ensemble de la solution réalisée 15
3.2 Parcours complet d'une demande 15
3.3 Administration et contrôle des accès 18
3.4 Assistance IA et validation humaine 18
3.5 Tests et validation 19
3.6 Limites et perspectives 19

Conclusion générale **21**

Références **22**

Liste des figures

1 Logo de SAFYRON ENGINEERING 2

2 Diagramme global des cas d'utilisation de la V1 7
3 Enregistrement et qualification des demandes 8
4 Composants applicatifs et systèmes externes de la V1 9
5 Organisation des responsabilités dans la conception backend 11
6 Entités persistantes et cardinalités du modèle de données V1 14

7 Connexion à l'application 15
8 Création et qualification initiale d'une demande 16
9 Liste filtrable des demandes 16
10 Traitement d'une demande affectée 17
11 Clôture après examen de la résolution 17
12 Gestion des utilisateurs internes 18
13 Assistance IA lors de la qualification 19

Liste des tableaux

1 Diagramme de Gantt simplifié — planning prévisionnel du stage 4

2 Capacités fonctionnelles de la V1 6

3 Dictionnaire de données synthétique 13

4 Niveaux de validation et portée 20

Liste des acronymes

API	Interface de programmation d'application (*Application Programming Interface*).
DTO	Objet de transfert de données (*Data Transfer Object*).
HTTP	Protocole de transfert hypertexte (*Hypertext Transfer Protocol*).
IA	Intelligence artificielle.
JPA	Interface de persistance Java (*Java Persistence API*).
JWT	Jeton web JSON (*JSON Web Token*).
RBAC	Contrôle d'accès fondé sur les rôles (*Role-Based Access Control*).
REST	Transfert d'état représentationnel (*Representational State Transfer*).
UML	Langage de modélisation unifié (*Unified Modeling Language*).

Introduction générale

Le stage professionnel s'est déroulé du 1er au 31 août 2026 au sein de SAFYRON ENGINEERING. Cette période d'un mois a été consacrée à un projet de génie logiciel lié au suivi des demandes techniques formulées par les clients de l'entreprise.

Dans les activités de l'entreprise, les demandes techniques formulées par les clients doivent être comprises, qualifiées et suivies jusqu'à l'achèvement de leur traitement. Chaque demande rassemble plusieurs informations utiles, notamment le client concerné, la nature du besoin, son niveau de priorité, la personne chargée du traitement et son état d'avancement. La continuité de ce suivi suppose que ces informations restent accessibles et cohérentes au fil des différentes étapes.

Le besoin général consiste ainsi à disposer d'un moyen commun pour centraliser les demandes, faciliter leur qualification, attribuer les responsabilités et conserver une trace de leur évolution. Il ne s'agit pas uniquement d'enregistrer une nouvelle demande, mais d'accompagner l'ensemble de son cycle de vie tout en maintenant un accès adapté aux responsabilités des utilisateurs internes.

La problématique retenue est donc la suivante : comment concevoir une solution interne cohérente permettant de centraliser, qualifier, affecter et suivre les demandes techniques des clients, tout en assurant la traçabilité de leur évolution et le contrôle humain des décisions importantes ?

Pour répondre à cette problématique, le projet propose une application web interne dédiée à la gestion des demandes techniques. Son objectif est de fournir un espace structuré pour leur prise en charge, depuis l'enregistrement jusqu'à la clôture. Une assistance fondée sur l'intelligence artificielle peut compléter la qualification, mais elle demeure facultative et ne remplace jamais la décision de l'utilisateur responsable.

Le rapport est organisé en trois chapitres. Le premier présente l'organisme d'accueil, le contexte, la problématique, les objectifs et la planification du stage. Le deuxième expose l'analyse des besoins et la conception de la solution. Le troisième décrit la réalisation, les mécanismes essentiels et les éléments de validation obtenus.

Chapitre 1

Contexte et cadrage du projet

Le cadre professionnel permet de situer l'origine du besoin et la problématique étudiée. La solution proposée, son périmètre et les objectifs retenus sont ensuite présentés, avant la démarche de travail et la planification du stage.

1.1 Organisme d'accueil et cadre du stage

Le stage professionnel s'est déroulé du 1er au 31 août 2026 au sein de SAFYRON ENGINEERING, située rue Lac Toba, aux Berges du Lac à Tunis. L'entreprise exerce dans le domaine des études techniques industrielles. Ses activités comprennent notamment la protection incendie, les études de dangers et l'analyse des risques industriels.

La figure 1 présente l'identité visuelle de l'organisme d'accueil.



FIGURE 1 – Logo de SAFYRON ENGINEERING

Le stage professionnel d'un mois a été réalisé dans le cadre de la cinquième année du cycle d'ingénieur en génie logiciel à EPI Digital School Sousse. Le travail a été mené individuellement, conformément aux exigences de l'école, sans rattachement à un service particulier. L'encadrement professionnel a été assuré par M. Achref Ammar, fondateur de SAFYRON ENGINEERING.

L'observation du contexte de l'entreprise a permis de mieux comprendre la nature des études réalisées et la place des demandes formulées par les clients. Sur la base de cette compréhension des processus de travail, l'étudiant a proposé l'idée d'une application web interne de gestion et de suivi des demandes techniques.

1.2 Contexte, existant et problématique

Les activités de SAFYRON ENGINEERING conduisent à traiter des demandes techniques provenant de clients. Une demande doit être comprise, orientée vers la personne concernée et suivie jusqu'à l'achèvement de son traitement.

L'étude de l'existant porte sur la logique générale de traitement des demandes, sans présumer des outils utilisés auparavant. À partir de cette compréhension du fonctionnement métier, plusieurs informations ont été retenues comme nécessaires au suivi dans l'application : l'identité du client, la nature du besoin, le niveau de priorité, la personne chargée du traitement, l'état courant et les changements successifs. Ces informations doivent rester cohérentes au cours de la prise en charge d'une demande.

Le besoin ne consiste donc pas seulement à enregistrer une demande. Le suivi doit permettre de la retrouver, de connaître son niveau d'avancement, d'identifier la personne responsable de son traitement et de conserver la trace des décisions importantes. Les intervenants disposent ainsi d'une vue claire de l'état de chaque demande et de la responsabilité associée à sa prise en charge.

La problématique du projet peut ainsi être formulée de la manière suivante : comment mettre à la disposition des intervenants internes un moyen cohérent pour centraliser, qualifier, affecter et suivre les demandes techniques des clients, tout en assurant la traçabilité de leur évolution et un accès adapté aux responsabilités de chacun ?

1.3 Solution proposée, objectifs et périmètre

Pour répondre à cette problématique, la solution proposée est une application web interne consacrée à la gestion et au suivi des demandes techniques. Elle fournit un espace commun dans lequel les demandes peuvent être enregistrées, qualifiées, affectées et suivies pendant leur traitement.

Les objectifs principaux sont de centraliser les demandes, de faciliter leur qualification et de formaliser leur cycle de vie. La solution doit également permettre d'affecter chaque demande, de suivre son traitement, de conserver un historique des changements et de protéger l'accès selon les responsabilités des utilisateurs. Une assistance fondée sur l'intelligence artificielle peut proposer une catégorie et une priorité ; elle reste facultative et toute proposition doit être validée par une personne avant son utilisation.

Les responsabilités internes se répartissent entre trois profils. Le Responsable technique pilote la prise en charge des demandes, l'Agent technique réalise leur traitement et l'Administrateur gère les comptes internes. Leurs interactions et leurs besoins sont détaillés dans l'analyse fonctionnelle.

Le périmètre retenu pour la V1 concerne les fonctions effectivement réalisées pour les utilisateurs internes. Le client y est représenté comme une entité métier associée à ses demandes,

mais il ne dispose ni d'un compte ni d'un portail. L'assistance par intelligence artificielle ne décide ni de la qualification ni de l'affectation : les choix finaux de catégorie et de priorité ainsi que les décisions de traitement restent sous contrôle humain. Ces limites correspondent à la durée du stage et distinguent la version réalisée des évolutions possibles.

1.4 Méthodologie et planification du travail

La méthodologie retenue repose sur une démarche progressive de génie logiciel : analyse du besoin et définition du périmètre, modélisation fonctionnelle, conception, réalisation, tests et validation. Cet enchaînement relie chaque étape aux besoins identifiés et permet de contrôler progressivement la cohérence du projet.

La durée d'un mois impose de privilégier la simplicité, une progression contrôlée et des validations régulières. Le périmètre de la première version a donc été limité à un ensemble cohérent de fonctions pouvant être analysées, réalisées et vérifiées pendant le stage. Le tableau 1 présente le planning prévisionnel retenu et sa répartition prévisionnelle en quatre périodes.

TABLE 1 – Diagramme de Gantt simplifié — planning prévisionnel du stage

Activité proposée	**S1** 01–07	**S2** 08–14	**S3** 15–21	**S4** 22–31
Compréhension du contexte de l'entreprise	■			
Analyse du besoin et définition du périmètre	■			
Analyse fonctionnelle et UML		■		
Conception de l'architecture et des données		■		
Réalisation backend et frontend			■	
Intégration progressive et finalisation			■	■
Tests et validation				■
Intégration de l'assistance IA et corrections finales				■

Le planning prévoit de consacrer la première semaine au contexte et au cadrage, la deuxième à l'analyse et à la conception, et la troisième principalement à la réalisation et au début de l'intégration. La dernière semaine est prévue pour finaliser la solution, conduire les vérifications et intégrer l'assistance IA ainsi que les corrections finales.

Cette répartition prévisionnelle organise les activités du stage sur quatre périodes au cours du mois.

Le périmètre retenu structure le traitement et le suivi des demandes techniques tout en laissant les décisions importantes aux utilisateurs concernés. Le chapitre suivant présente l'analyse des besoins ainsi que la conception fonctionnelle et technique.

Chapitre 2

Analyse et conception

Introduction

La conception transforme les besoins du stage en responsabilités, règles et structures cohérentes. Elle part des utilisateurs internes et du cycle de vie d'une demande, puis établit l'architecture, les mécanismes de sécurité, l'intégration de l'assistance IA et le modèle de données. Chaque choix est ainsi relié au problème qu'il doit résoudre.

2.1 Spécification des besoins

Cette section précise les personnes concernées par la solution et les attentes auxquelles elle doit répondre. Elle distingue les fonctions attendues des qualités générales du système, avant d'aborder les choix de conception.

2.1.1 Acteurs et responsabilités

L'application distingue trois rôles internes, qu'une même personne peut cumuler lorsque l'organisation l'exige. Le Client reste extérieur à cette répartition des accès.

- **Responsable technique :** il pilote la demande depuis son enregistrement et sa qualification jusqu'à la décision de clôture.
- **Agent technique :** il prend en charge les demandes qui lui sont affectées et documente leur traitement puis leur résolution.
- **Administrateur :** il maintient les comptes des utilisateurs internes et leurs rôles.
- **Client :** il constitue une entité métier associée à la demande, et non un acteur utilisateur de la V1 ; aucun portail ni droit d'accès client n'est introduit.

2.1.2 Besoins fonctionnels

Le tableau 2 synthétise les grandes fonctions attendues de la V1, depuis l'accès interne jusqu'à l'assistance à la qualification.

TABLE 2 – Capacités fonctionnelles de la V1

Capacité	Comportement attendu
Accès interne	Authentifier un utilisateur et orienter son accès selon ses rôles.
Gestion des demandes	Enregistrer, consulter, filtrer et suivre les demandes techniques.
Qualification	Définir une catégorie et une priorité validées par le Responsable technique.
Workflow	Affecter, traiter, résoudre, clôturer, refuser ou annuler selon l'état courant.
Administration	Gérer les comptes, l'état actif et les rôles métier autorisés.
Assistance IA	Proposer une catégorie et une priorité, sans décision automatique.

Ces besoins décrivent le résultat attendu sans anticiper les mécanismes techniques retenus pour y répondre.

2.1.3 Besoins non fonctionnels

- **Sécurité :** chaque utilisateur accède uniquement aux fonctions autorisées par ses responsabilités.
- **Cohérence :** les informations d'une demande restent compatibles avec son évolution.
- **Traçabilité :** les changements importants intervenus pendant le traitement doivent pouvoir être compris.
- **Maintenabilité :** la solution doit pouvoir évoluer sans rendre ses responsabilités difficiles à comprendre ou à modifier.
- **Ergonomie :** l'état d'une demande doit être identifiable rapidement, y compris sur différentes tailles d'écran.
- **Continuité de service IA :** l'indisponibilité du fournisseur ne doit jamais empêcher une qualification manuelle.

2.2 Conception fonctionnelle

La conception fonctionnelle décrit les interactions attendues sans anticiper leur réalisation technique. Le diagramme global situe d'abord les responsabilités ; les cas majeurs et leurs scénarios précisent ensuite le comportement attendu.

La figure 2 présente les acteurs internes et les capacités du système auxquelles ils participent.

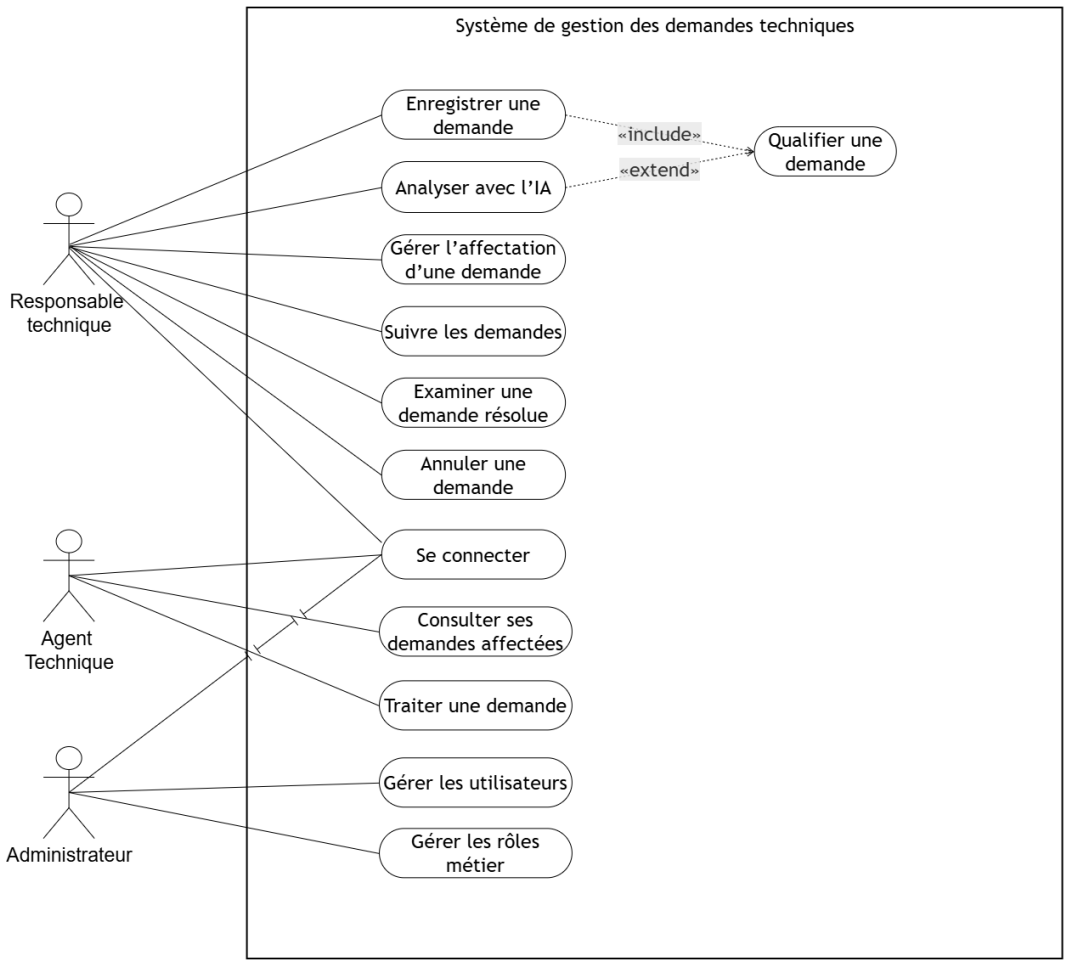


FIGURE 2 – Diagramme global des cas d'utilisation de la V1

Le diagramme distingue les responsabilités essentielles : le Responsable technique pilote le cycle de la demande, l'Agent traite les demandes qui lui sont affectées et l'Administrateur gère les utilisateurs internes. L'assistance IA reste facultative et sous le contrôle du Responsable technique.

Enregistrement et qualification. Le scénario nominal sélectionne un client existant ; l'alternative permet de saisir les informations minimales d'un nouveau client avant de valider la catégorie et la priorité.

Traitement et résolution. L'Agent documente la résolution, puis le Responsable technique l'accepte ou la refuse. Un refus replace la demande en `EN_COURS`, réinitialise la date de résolution et reste tracé.

Réaffectation et annulation. Une demande réaffectée après le début du traitement revient à `ASSIGNEE` sans perte des informations saisies. L'annulation avant clôture exige un motif.

Assistance IA. L'analyse du titre et de la description propose une catégorie et une priorité modifiables, sans décision automatique et sans bloquer la qualification manuelle.

La figure 3 détaille le scénario d'enregistrement et ses deux alternatives selon que le client existe déjà ou doit être renseigné.

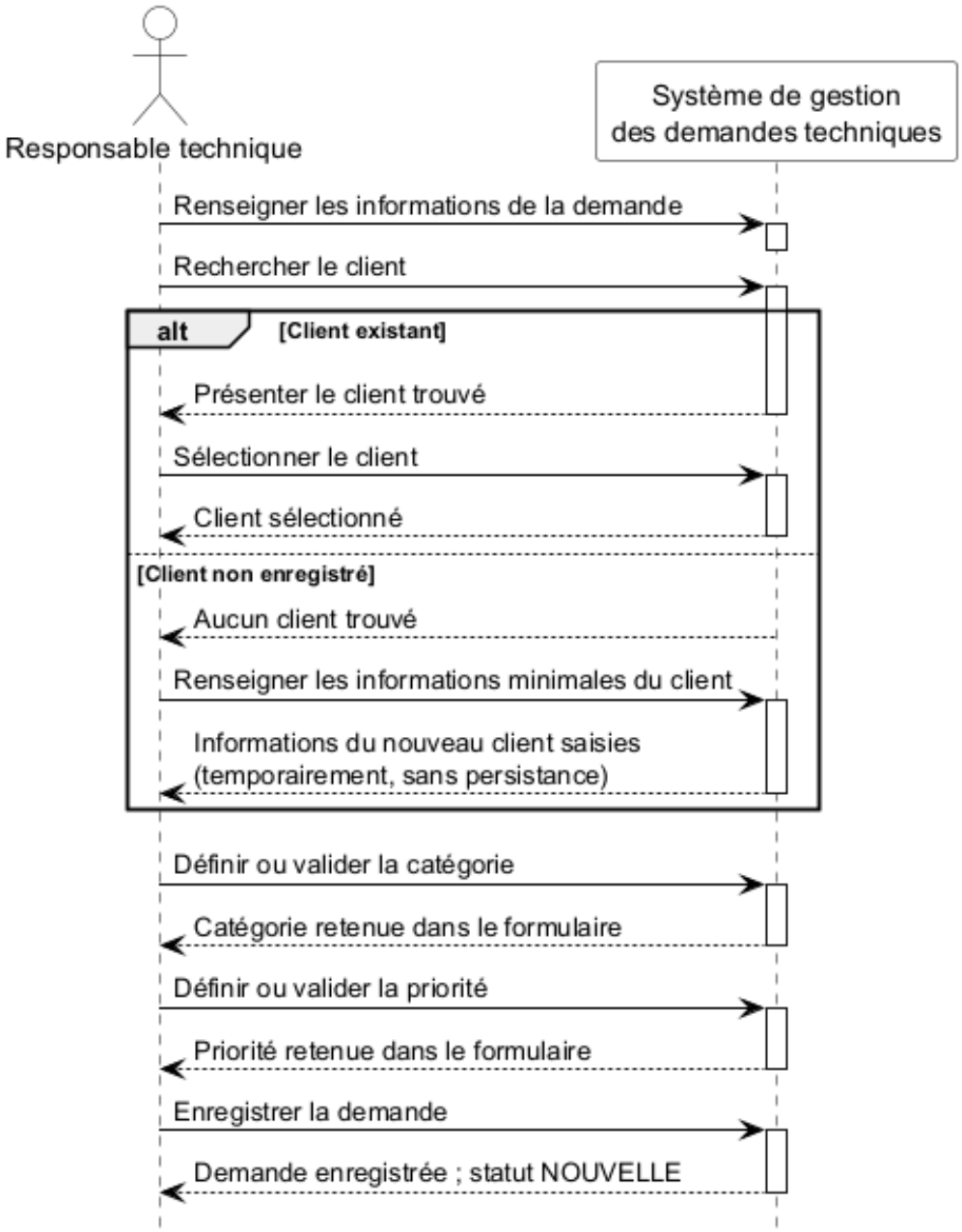


FIGURE 3 – Enregistrement et qualification des demandes

La séquence montre que le Responsable technique peut sélectionner un client existant ou renseigner un nouveau client dans le même parcours. La catégorie et la priorité sont validées avant l'enregistrement final, qui préserve la cohérence entre le client, la demande et l'événement de création.

2.3 Conception technique

Cette section traduit les besoins précédents en organisation technique. Elle présente les frontières de l'application, les principes de conception retenus, la protection des accès et l'intégration de l'assistance intelligente.

2.3.1 Architecture générale

L'architecture doit séparer l'interaction utilisateur, les règles métier, la persistance relationnelle et le fournisseur IA, tout en conservant un déploiement simple pour la V1. La figure 4 présente les frontières retenues.

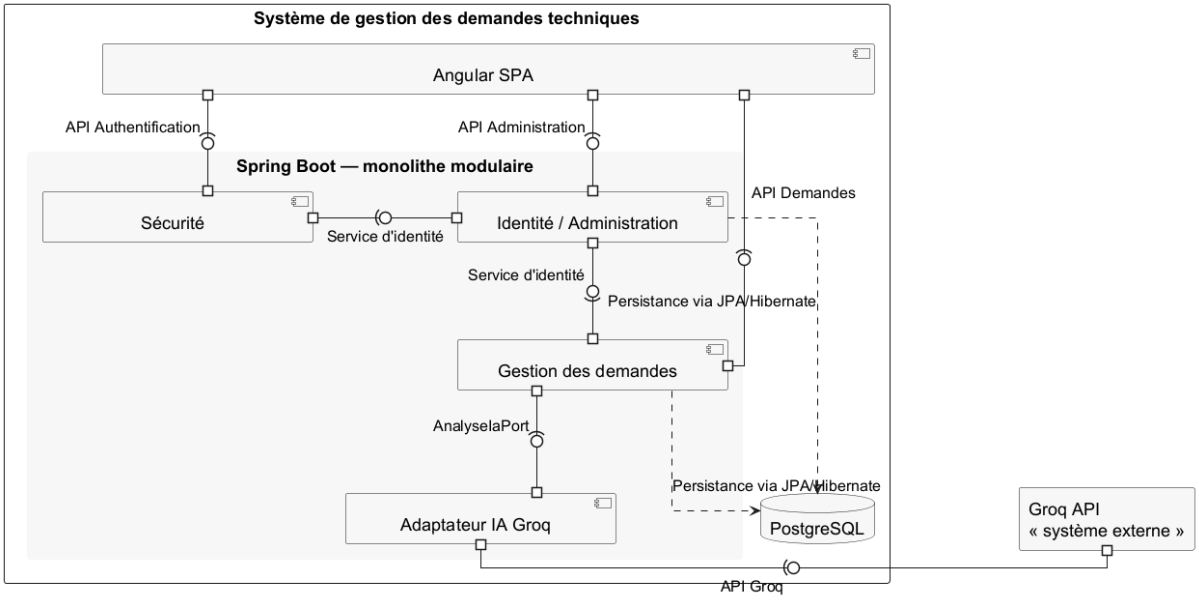


FIGURE 4 – Composants applicatifs et systèmes externes de la V1

Le diagramme sépare l'interface Angular, le backend Spring Boot et la persistance PostgreSQL. Le backend reste organisé en monolithe modulaire afin de distinguer les responsabilités sans multiplier les déploiements, tandis que l'intégration Groq est isolée du cœur métier derrière une frontière dédiée.

2.3.2 Principes de conception et patterns

Les choix retenus répondent à des besoins précis sans permettre d'affirmer que tous les principes SOLID sont appliqués uniformément.

Responsabilité unique (SRP). La séparation des rôles rend les composants plus lisibles ; contrôleurs, services, domaine et persistance gardent chacun une responsabilité principale identifiable (Martin 2017).

Inversion des dépendances (DIP) et Port/Adapter. `AnalyseIaPort` exprime le besoin du cœur applicatif, tandis que `GroqAnalyseIaAdapter` réalise l'appel externe. Cette séparation limite l'impact d'un changement de fournisseur (Martin 2017 ; Gamma et al. 1994).

Repository. Ce pattern isole l'accès aux données des services métier (Fowler 2002).

DTO. Les objets de transfert séparent les contrats HTTP des entités persistantes, sans être présentés comme une solution universelle.

State Pattern. Il n'est pas retenu pour la V1, car un enum de statut et des opérations métier explicites suffisent au cycle actuel. Ce choix pourrait être réévalué si le workflow s'étend fortement (Gamma et al. 1994).

La figure 5 affine la vue générale en montrant comment les responsabilités et ces choix de conception sont répartis à l'intérieur du backend.

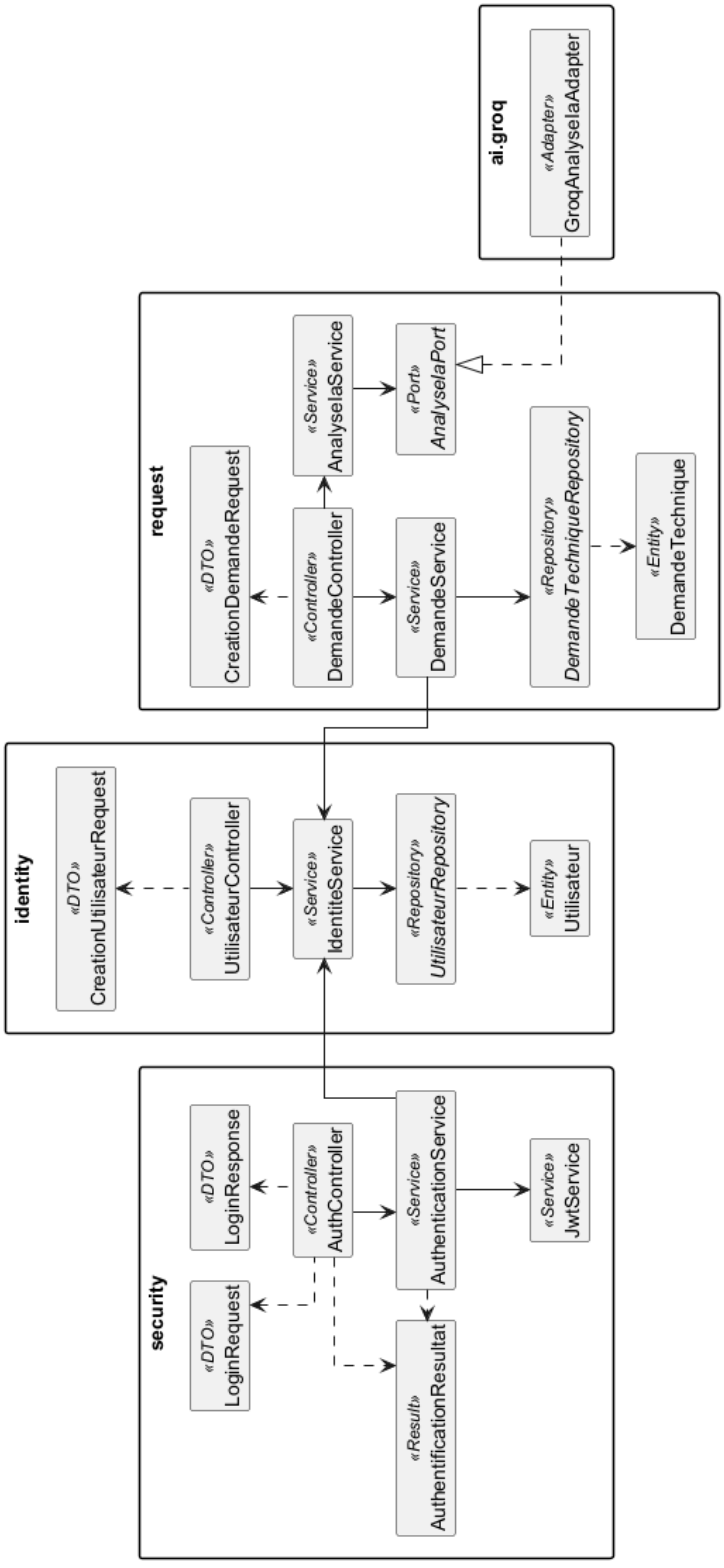


FIGURE 5 – Organisation des responsabilités dans la conception backend

Le diagramme précise que les contrôleurs portent la frontière HTTP, que les services orchestrent les cas d'utilisation et que le domaine protège les invariants. Les repositories isolent la persistance, tandis que le port et son adaptateur séparent l'intégration Groq du cœur applicatif.

2.3.3 Sécurité de la solution

L'**authentification** vérifie l'identité et l'état actif du compte avant l'émission d'un JSON Web Token (JWT) de durée limitée, transmis avec les requêtes protégées (Jones, Bradley et Sakimura 2015).

L'**autorisation** repose sur le contrôle d'accès basé sur les rôles (RBAC), complété par des contrôles contextuels liés à l'affectation, à l'état de la demande et à l'interdiction d'auto-désactivation de l'Administrateur.

Les **mots de passe** sont hachés avec BCrypt et les secrets proviennent d'une configuration externe. La conservation temporaire de la session dans le navigateur facilite le rechargement, mais ne vaut pas autorisation : le backend vérifie le JWT, l'état actif, les rôles courants et l'action demandée, et reste l'autorité.

2.3.4 Assistance intelligente

Le service analyse le titre et la description de la demande afin de proposer une catégorie et une priorité. Cette proposition reste soumise à une validation humaine, et la qualification manuelle demeure disponible en cas d'indisponibilité externe.

2.4 Conception de la base de données

La base relationnelle doit préserver l'identité des utilisateurs, le contenu et le cycle des demandes, puis la trace des événements. Le dictionnaire précise d'abord les principales données et leurs contraintes ; le modèle d'entités montre ensuite leurs relations.

TABLE 3 – Dictionnaire de données synthétique

Entité	**Attribut**	**Type**	**Contrainte et signification**
Utilisateur	id	Long / BIGINT	Clé primaire, séquence.
Utilisateur	email	String / VARCHAR	Obligatoire, normalisé et unique.
Utilisateur	actif	boolean / BOOLEAN	Obligatoire ; compte autorisé ou désactivé.
Utilisateur	roles	Set<Role>	Association ; plusieurs rôles possibles.
Client	id	Long / BIGINT	Clé primaire, séquence.
Client	nom, email, telephone	String / VARCHAR	Trois informations obligatoires ; email non unique.
DemandeTechnique	reference	String / VARCHAR	Obligatoire, unique et immuable ; référence métier.
DemandeTechnique	titre	String / VARCHAR	Obligatoire ; intitulé de la demande.
DemandeTechnique	description	String / TEXT	Obligatoire ; détail de la demande.
DemandeTechnique	categorie	Categorie / VARCHAR	Obligatoire ; valeur de la liste V1.
DemandeTechnique	priorite	Priorite / VARCHAR	Obligatoire ; BASSE à CRITIQUE.
DemandeTechnique	statut	StatutDemande / VARCHAR	Obligatoire ; état du cycle de vie.
DemandeTechnique	agentAffecte	Utilisateur / FK	Optionnel ; au plus un Agent affecté.
DemandeTechnique	dates métier	Instant / timestamptz	Création et modification obligatoires ; résolution, clôture et annulation optionnelles.
HistoriqueDemande	typeEvenement	String / VARCHAR	Obligatoire ; nature du changement.
HistoriqueDemande	ancienne/nouvelle valeur	String / TEXT	Optionnelles ; valeurs avant et après.
HistoriqueDemande	demande, auteur	FK	Obligatoires ; origine et responsable de l'événement.

Le dictionnaire constitue la synthèse principale des attributs, types et contraintes. La figure 6 montre ensuite leurs relations et cardinalités.

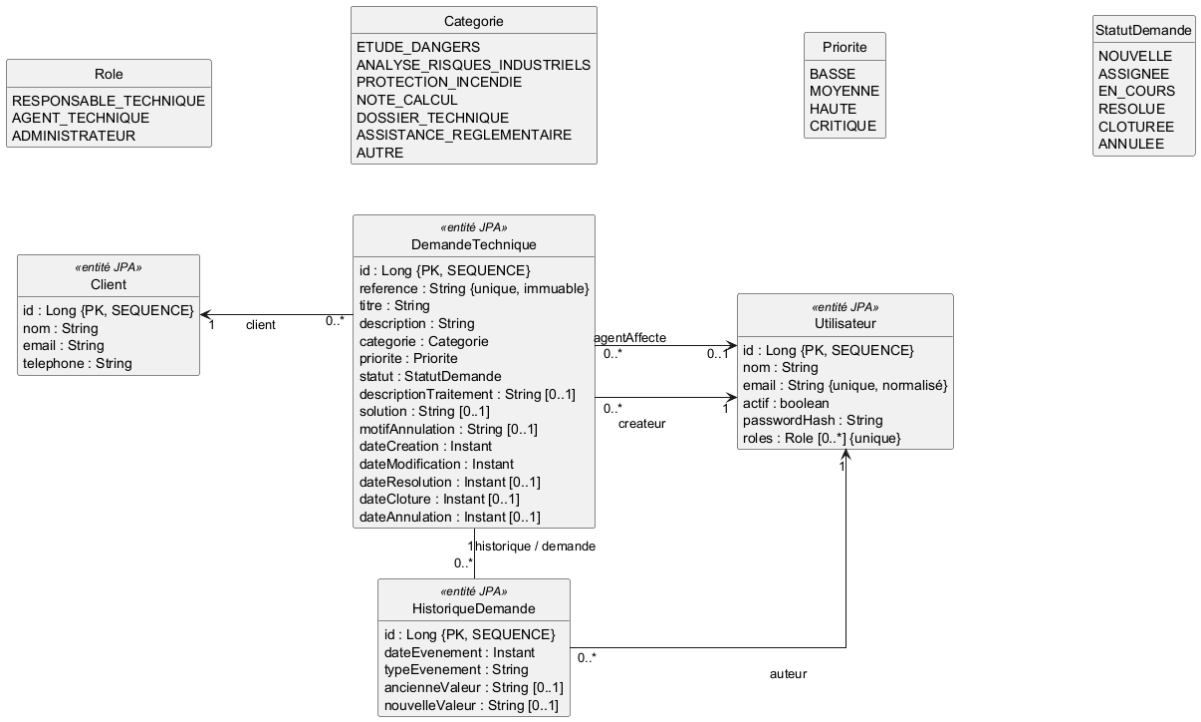


FIGURE 6 – Entités persistantes et cardinalités du modèle de données V1

`DemandeTechnique` constitue l'entité centrale : elle relie le Client, le créateur et, lorsqu'il existe, l'Agent affecté. `Utilisateur` porte l'identité et les rôles des acteurs internes, tandis que `Client` conserve les informations de la personne concernée par la demande.

`HistoriqueDemande` assure la traçabilité des changements et de leur auteur. Les instants sont stockés en `TIMESTAMP WITH TIME ZONE`, en cohérence avec le type `Instant` côté Java (PostgreSQL Global Development Group 2026). Les énumérations contrôlent les rôles, catégories, priorités et statuts autorisés sans devenir des entités indépendantes.

Conclusion

Ce chapitre a traduit les besoins en conception fonctionnelle, en architecture et en modèle de données. Il a également précisé la protection des accès et l'intégration d'une assistance IA soumise à validation humaine. Le chapitre suivant présente la réalisation de cette conception et les éléments qui permettent de la valider.

Chapitre 3

Réalisation et validation

3.1 Vue d'ensemble de la solution réalisée

Angular fournit l'application web et les parcours utilisateur. Spring Boot expose l'API et applique les règles métier. PostgreSQL conserve les données et l'historique. Cette répartition permet de faire évoluer l'interface sans déplacer les invariants métier dans le navigateur.

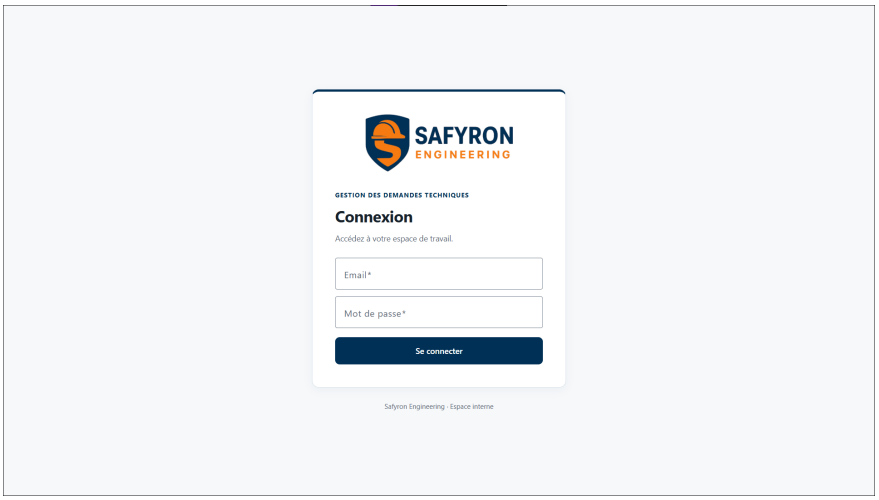


FIGURE 7 – Connexion à l'application

Après authentification, le frontend conserve le token et le snapshot minimal utile à l'orientation de l'interface. Les autorisations effectives restent décidées par l'API.

3.2 Parcours complet d'une demande

Le parcours commence par la création et la qualification. Le Responsable technique renseigne un titre, une description, un client, une catégorie et une priorité. Il peut demander une suggestion IA, mais garde la possibilité de modifier les valeurs avant l'enregistrement.

La liste permet ensuite de retrouver les demandes par filtres, tri et pagination côté serveur. Après affectation, l'Agent traite la demande, renseigne la solution et déclare la résolution. Le

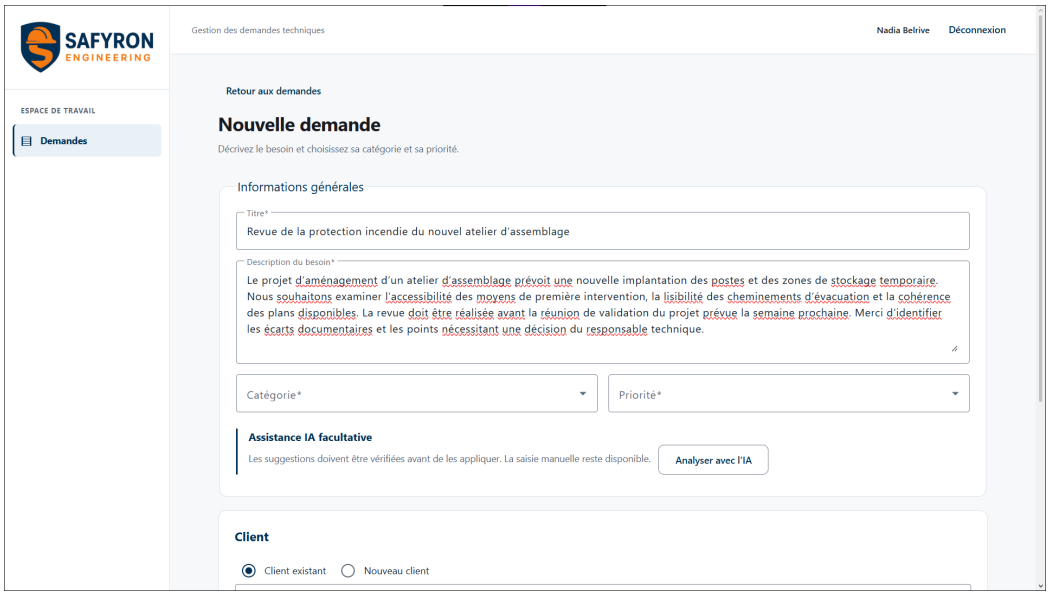


FIGURE 8 – Création et qualification initiale d'une demande

Responsable examine cette proposition : il clôture la demande ou la refuse pour demander une reprise.

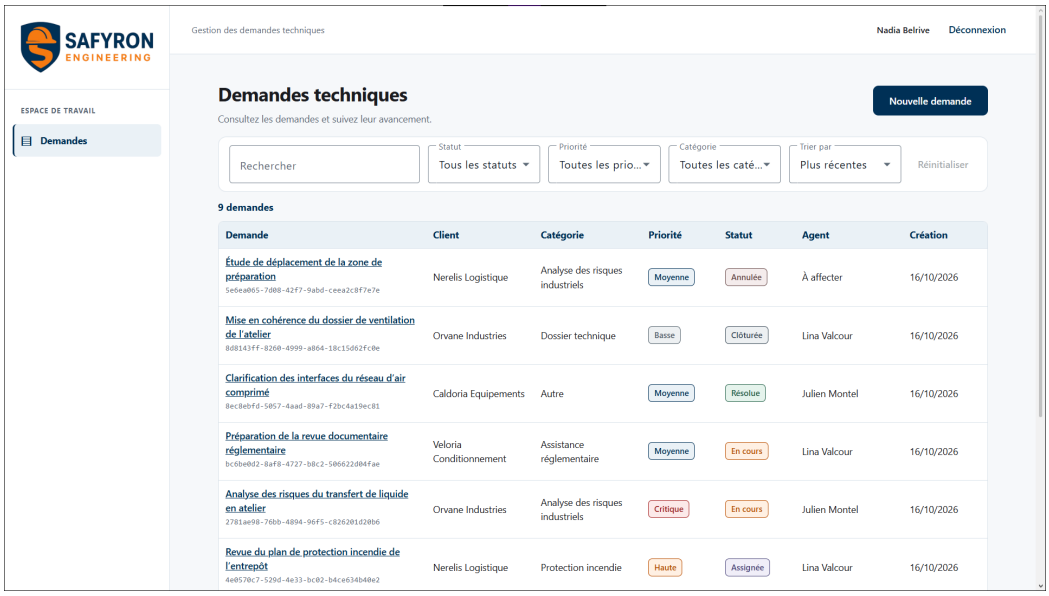


FIGURE 9 – Liste filtrable des demandes

Ces écrans illustrent une propriété importante : l'interface présente des actions dépendantes du rôle et du statut, mais cette présentation ne remplace pas le contrôle côté serveur.

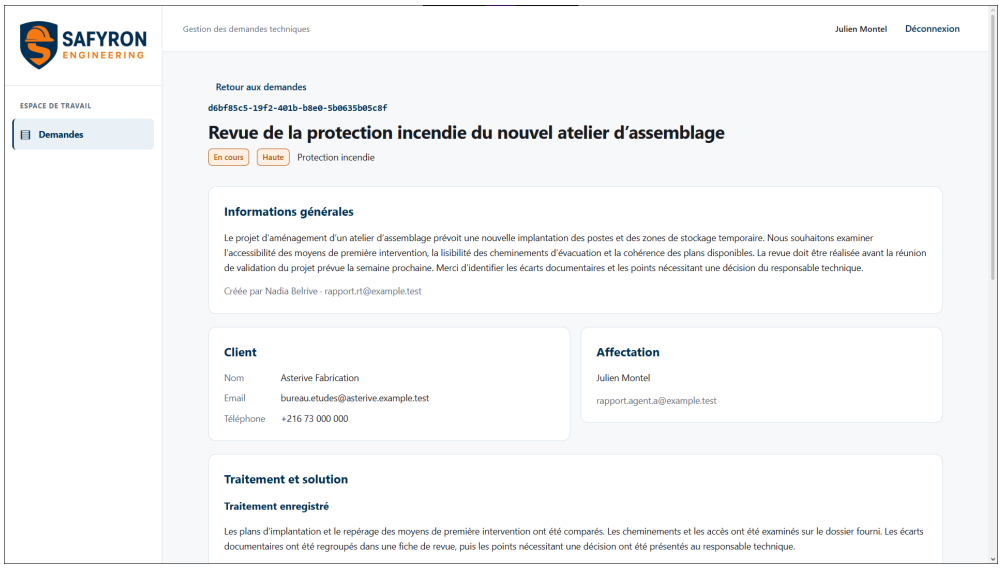


FIGURE 10 – Traitement d'une demande affectée

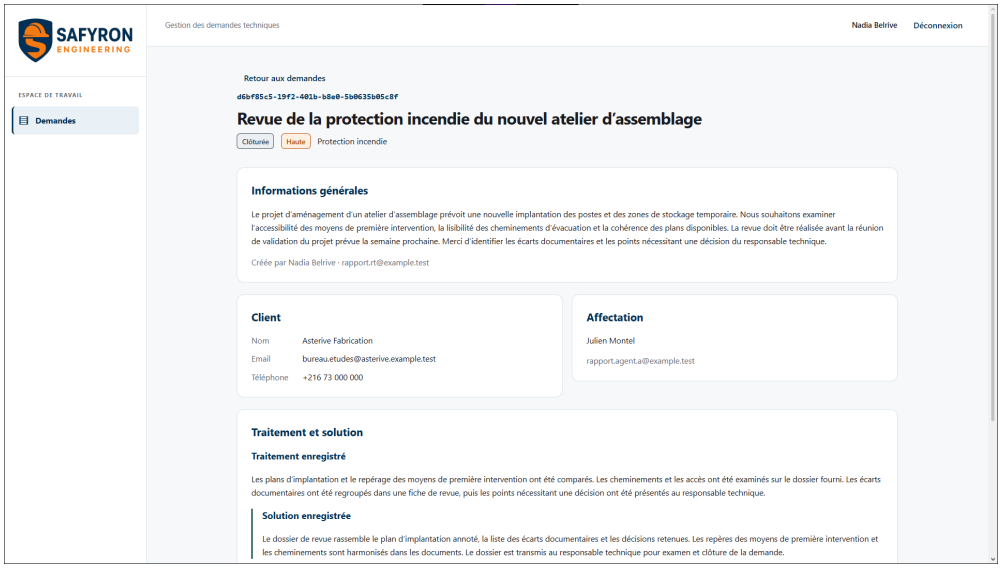


FIGURE 11 – Clôture après examen de la résolution

3.3 Administration et contrôle des accès

L'administration permet de créer et modifier les comptes, de gérer leur état actif et de remplacer leurs rôles métier. L'auto-désactivation du compte courant est refusée par l'API avec une réponse 409 et l'interface conserve un retour compréhensible.

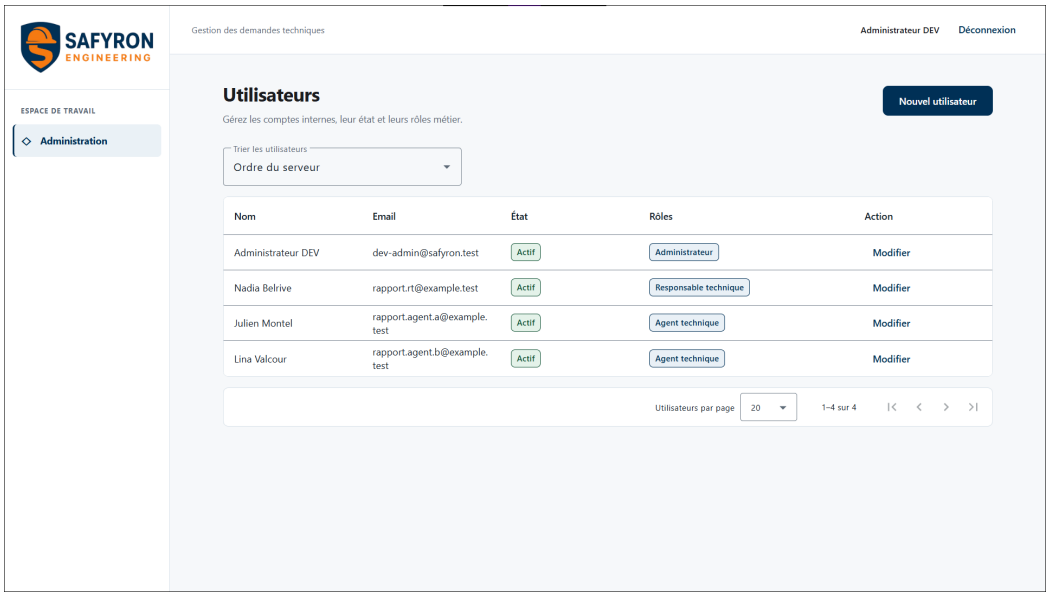


FIGURE 12 – Gestion des utilisateurs internes

Le backend relit l'utilisateur et ses rôles à chaque requête protégée. Un rôle supprimé ou un compte désactivé ne conserve donc pas ses droits simplement parce qu'un snapshot ancien existe dans le navigateur.

3.4 Assistance IA et validation humaine

L'analyse IA est déclenchée depuis le formulaire de nouvelle demande. Le frontend envoie uniquement le titre et la description à l'endpoint backend. Le serveur appelle l'adaptateur Groq via ‘AnalyseIaPort‘, contrôle la réponse puis renvoie une proposition temporaire. La catégorie et la priorité ne sont jamais appliquées automatiquement.

Une modification du titre ou de la description invalide la suggestion. Lorsque la clé Groq est absente, l'API renvoie une indisponibilité contrôlée et le formulaire reste utilisable manuellement. Le benchmark exploratoire porte sur 24 exemples : sorties valides dans les 24 cas, catégorie correcte dans 23 cas et priorité dans 22 cas. Ces valeurs ne permettent pas d'annoncer une performance générale ni une supériorité universelle d'un fournisseur.

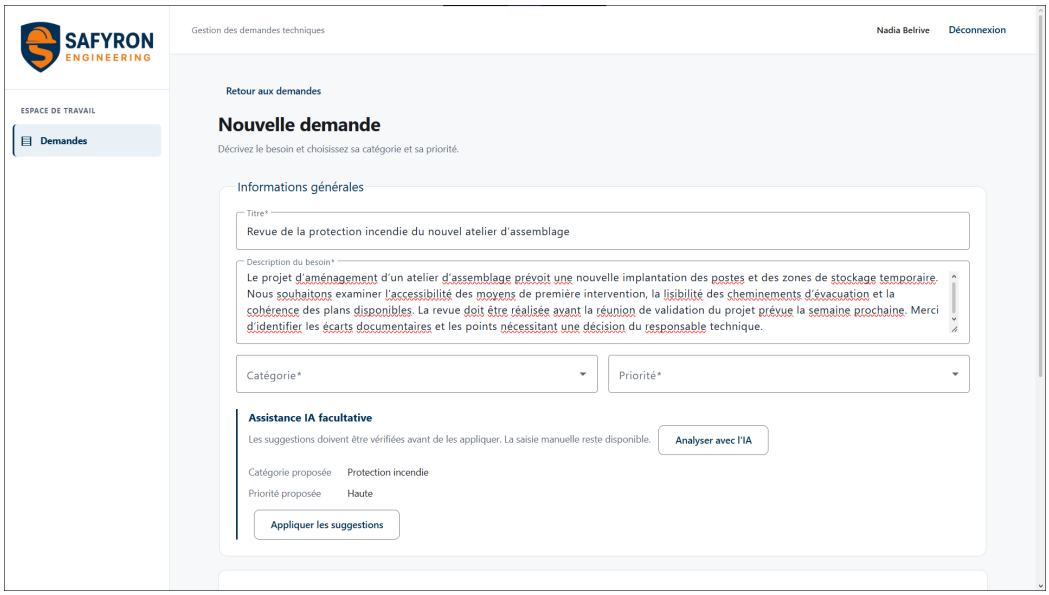


FIGURE 13 – Assistance IA lors de la qualification

3.5 Tests et validation

La validation backend atteint 270 tests réussis avec ‘clean test‘ et ‘clean verify‘, sans échec, erreur ou test ignoré. Les tests IA hors ligne utilisent H2 et un stub HTTP local ; ils ne constituent pas un appel Groq. Le frontend a été vérifié par ‘npm ci‘, ‘npm run build‘ et ‘npm test – –watch=false‘, avec 184 tests réussis dans 15 fichiers pour la clôture IA.

La validation fonctionnelle humaine a couvert la connexion, la liste, la création, la sélection client, les transitions de demande, l’administration, l’auto-désactivation et l’assistance IA réelle en local. Une revue responsive et accessibilité a également été effectuée sur plusieurs tailles. Ces observations ne remplacent pas une campagne de test de charge ou une validation de production.

3.6 Limites et perspectives

Le rapport ne prétend pas mesurer la performance en production, la résistance à la charge ou la qualité IA sur un corpus représentatif du long terme. Le périmètre reste interne et le client n’interagit pas directement avec la plateforme. La session navigateur utilise ‘sessionStorage‘, ce qui permet une restauration après rechargement mais conserve les risques propres à un stockage accessible au JavaScript du navigateur.

Les perspectives cohérentes avec la V1 sont l’élargissement du corpus d’évaluation IA, l’ajout de mesures répétées et l’amélioration de l’observabilité. Toute extension du workflow, du portail client ou du déploiement devra être précédée d’une analyse de besoin et d’une validation dédiée.

TABLE 4 – Niveaux de validation et portée

Validation	**Ce qu'elle permet d'affirmer**
Tests backend	Les comportements couverts par 270 tests réussissent dans l'environnement vérifié.
Tests frontend	La construction et les scénarios automatisés couverts réussissent.
Intégration locale IA	Le parcours Angular vers Spring Boot et Groq a fonctionné dans les conditions observées.
Validation humaine	Les parcours et l'ergonomie vérifiés ne présentent pas de blocage constaté dans le périmètre revu.
Benchmark IA	Résultats exploratoires sur 24 exemples, non généralisables.

Conclusion générale

Le stage a conduit à une application V1 couvrant l'enregistrement, la qualification, l'affectation, le traitement et le suivi des demandes techniques. La séparation entre interface Angular, API Spring Boot et persistance PostgreSQL fournit une base lisible et évolutive. La sécurité combine authentification JWT, rôles métier et contrôles contextuels.

L'assistance IA facultative via Groq propose une catégorie et une priorité, mais la décision finale reste modifiable et validée par le Responsable technique. Les tests automatisés et les validations humaines montrent que le périmètre V1 est fonctionnel dans les conditions vérifiées.

Les limites concernent le périmètre interne, l'absence de portail client et le caractère exploratoire du benchmark IA. Des travaux ultérieurs pourraient élargir le corpus d'évaluation, renforcer l'observabilité et étudier les besoins de déploiement, sous réserve de validations supplémentaires.

Références

Fowler, Martin (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional. isbn : 9780321127426.

Gamma, Erich et al. (1994). *Design Patterns : Elements of Reusable Object-Oriented Software*. 1re éd. Addison-Wesley Professional. isbn : 9780201633610.

Jones, Michael, John Bradley et Nat Sakimura (mai 2015). *JSON Web Token (JWT)*. url : https://www.rfc-editor.org/rfc/rfc7519.html (visité le 11/10/2026).

Martin, Robert C. (2017). *Clean Architecture : A Craftsman's Guide to Software Structure and Design*. 1re éd. Pearson. isbn : 9780134494166.

PostgreSQL Global Development Group (2026). *Date/Time Types*. url : https://www.postgresql.org/docs/current/datatype-datetime.html (visité le 11/10/2026).